



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

14 мая 2025 г.

Цель работы

Изучение эффективных приёмов организации приложений и оптимизации вызовов OpenGL. Задача заключается в оптимизации OpenGL-приложения, разработанного ранее, путём внедрения как минимум четырёх различных методов оптимизации и анализе их эффективности на основе измерений производительности.

Теория

Оптимизация в OpenGL является ключевым элементом повышения производительности графических приложений. Наиболее часто используемые техники включают:

- **Дисплейные списки (Display Lists)** — позволяют один раз скомпилировать команды рендеринга и затем быстро их выполнять.
- **Массивы вершин (Vertex Arrays)** — позволяют передавать массивы вершин, нормалей и текстурных координат одним вызовом.
- **Использование индексов (Element Buffer Object)** — сокращает дублирование вершин за счёт переиспользования данных.
- **glColorMaterial** — упрощает управление цветом и освещением без постоянного обновления свойств материала.

Реализация

В качестве объекта для оптимизации использовалась модель тора, реализованная в файле Torus.py. Основной рендеринг происходил с помощью метода draw(), в котором каждый кадр отрисовывался с нуля. Были реализованы следующие оптимизации:

1. Использование дисплейных списков

При активации флага `OPTIMIZE_USE_DL = True` создаётся дисплейный список, который компилирует команды рисования тора один раз. В дальнейшем используется вызов `glCallList`, что значительно ускоряет рендеринг.

2. Использование массивов вершин (Vertex Arrays)

При флаге `OPTIMIZE_USE_VA = True` используется метод `draw_vertices()`, который работает с массивами вершин, нормалей и текстурных координат. Эти массивы генерируются заранее с помощью метода `generate()`.

3. Использование индексированных массивов (EBO)

Метод `generate2()` генерирует индексы, позволяющие переиспользовать вершины при рендеринге. Метод `draw_vertices_indices()` использует `glDrawElements`, что сокращает объём передаваемых данных.

4. Оптимизация с помощью glColorMaterial

Позволяет использовать цвет напрямую вместо полного описания материала при включённом освещении, уменьшая накладные расходы при вызовах `glMaterial`.

Результаты

Таблица 1: FPS по отдельным оптимизациям

Оптимизация	FPS	Разница с базой
Без оптимизаций	204.0	—
Display Lists	4883.9	+4680
Vertex Arrays	3992.8	+3788
EBO	197.1	−7
glColorMaterial	201.1	−3

Таблица 2: FPS при комбинированных оптимизациях

Оптимизация	FPS	Разница с базой
Без оптимизаций	188.4	—
+Display Lists	4964.6	+4776
+Vertex Arrays	5144.5	+180
+glColorMaterial	5277.8	+133
+EBO	5183.7	−94

Вывод

Оптимизации, основанные на дисплейных списках и массивах вершин, оказались наиболее эффективными. Использование индексов (EBO) и glColorMaterial в отдельных случаях не давало значительного прироста, а иногда даже снижало производительность. Наилучший результат достигался при последовательном включении всех оптимизаций, кроме EBO, что позволило повысить FPS почти в 28 раз по сравнению с начальным вариантом.